A PROJECT REPORT

AI Email Assistant

A practical tool for understanding email and preparing replies

Abstract

An inbox can be difficult to scan when a deadline, a routine update, and a message that needs no reply all arrive together. The AI Email Assistant brings message review and reply drafting into one small web application. It labels messages, estimates priority, explains that assessment in plain terms, and gives the user a draft they can edit before deciding what to do.

The interface is built with React and Vite. A FastAPI backend validates requests with Pydantic and uses LangChain ChatGroq for live analysis and draft generation. The analysis response includes a category, a priority level, a short summary, reasons, and possible action items. Drafts can be requested in Professional, Formal, or Friendly tone. Email sending is handled separately through SMTP and remains in dry-run mode by default.

The current application is a local demonstration rather than a connected mailbox service. Its sample and user-added messages are stored in browser localStorage; it does not connect to Gmail or Outlook, provide multi-user authentication, or encrypt that browser storage. The project includes backend tests for API validation and delivery behavior, and its frontend production build has been verified. The system is useful as a working prototype, provided that a person reviews the original message and any AI-generated text before acting.

Keywords: email triage, FastAPI, React, Groq, reply drafting, SMTP

Table of Contents

Abstract 2
Chapter 1: Introduction 4
1.1 Background and motivation 4
1.2 Problem and project aim 4
1.3 Objectives and scope 4
Chapter 2: Requirements and design 5
2.1 User workflow 5
2.2 System architecture 5
Chapter 3: Implementation 6
3.1 Frontend and API 6
3.2 AI analysis and drafting 6
3.3 Email delivery 6
Chapter 4: Testing and results 7
4.1 Verification approach 7
4.2 Results and observations 7
Chapter 5: Limitations and conclusion 8
5.1 Current limits 8
5.2 Future work and conclusion 8
References 9

Abbreviations

Term	Meaning
API	Application programming interface
CORS	Cross-origin resource sharing
SMTP	Simple Mail Transfer Protocol
UI	User interface

Chapter 1: Introduction

1.1 Background and motivation

Email is still where many people receive requests, decisions, and reminders, but an inbox does not make it easy to tell which message deserves attention first. Even a short message can take time to understand, especially when the next step or deadline is buried in the body. This project explores a straightforward way to make that first review quicker without removing the person from the decision.

1.2 Problem and project aim

The project addresses two repeated tasks: sorting messages into a useful order and finding a sensible starting point for a reply. Its aim is to combine those tasks in one interface. The assistant can suggest a category and priority, explain the reasons it found in the message, and prepare reply text. A user can review and edit the result before taking action.

1.3 Objectives and scope

- Present messages with sender, subject, preview, time, category, unread state, and priority cues.
- Use AI to produce a category, priority, concise explanation, summary, and possible action items.
- Offer editable drafts in three supported tones and provide copy and regenerate actions.
- Keep the sending action explicit, with dry-run behavior as the default.
- Build a working local application with a React/Vite interface and a FastAPI service.

The scope is a local prototype. The project does not claim a production mailbox connection, clinical or business-grade accuracy, or a guarantee that AI output is correct.

Chapter 2: Requirements and design

2.1 User workflow

The workflow follows the way a person already handles a message. They scan or filter the inbox, open a message, read the AI priority explanation, request a reply in a chosen tone, then review and edit that text. Copying or regenerating a draft does not send it. Sending is a separate action, and the backend reports whether it actually sent a message or only completed a dry run.

Step	Person's action	Application response
1. Find	Open Inbox or Priority and use a category filter.	Shows message details and visual unread or priority cues.
2. Review	Open a message and read its content.	Shows the available category, priority, and explanation.
3. Draft	Select Professional, Formal, or Friendly and request a reply.	Returns editable draft text from the backend.
4. Decide	Edit, copy, regenerate, or choose Send.	Shows a dry-run result or a confirmed SMTP result.

2.2 System architecture

The browser runs the React application through Vite [2, 3]. During local development, Vite forwards requests under /api to the FastAPI server on port 8000. The backend validates incoming JSON with Pydantic and FastAPI [1, 5], calls the AI service for analysis or drafting, and returns typed JSON responses. Email delivery follows a separate SMTP service. The browser stores the demo inbox and drafts in localStorage.

Part	Role in the application
React and Vite	Inbox, message detail, draft editing, and local development server.
FastAPI and Pydantic	HTTP routes, request validation, response schemas, and API documentation.
LangChain ChatGroq	Structured email analysis and tone-aware reply generation.
Browser localStorage	Persistence for the local demo inbox, sent items, and theme.
SMTP	Optional server-side email delivery; dry-run is enabled by default.

Chapter 3: Implementation

3.1 Frontend and API

The React interface presents Inbox, Priority, Sent, and Settings views [2]. It includes category filters, unread indicators, priority markers, a message detail view, and an editor for reply drafts. Users can add messages to the local demo inbox. The frontend sends real requests for AI analysis, drafting, and delivery; the API base can be configured for a separate backend, while local development uses the Vite proxy [3].

Route	Purpose	Typical result
GET /api/health	Check backend availability.	JSON status response.
POST /api/analyze	Classify and assess a message.	Category, priority, reasons, summary, actions.
POST /api/draft	Prepare a reply in a selected tone.	Validated draft text.
POST /api/send	Request SMTP delivery.	Sent or dry_run status.

3.2 AI analysis and drafting

The backend sends email content to the configured Groq service when a user requests AI analysis or a draft, using LangChain's ChatGroq integration [4, 6]. Pydantic models constrain the accepted categories, priority levels, tones, and result fields [5]. The prompts treat email text as untrusted content, ask the model not to follow instructions embedded in a message, and avoid inventing deadlines or commitments. Those safeguards reduce predictable mistakes, but they cannot guarantee that every model response is correct.

The Groq API key and model setting are read by the backend from its environment, including backend/.env. The frontend does not need the key. Missing credentials return a service-unavailable response; rate limits and other provider failures are mapped to specific safe errors rather than exposing secrets or raw provider responses.

3.3 Email delivery

Sending is intentionally separate from drafting. With SEND_DRY_RUN=true, the endpoint returns a dry-run status and no email is transmitted. To send through SMTP, an operator must configure the server settings and deliberately disable dry-run. Python's smtplib provides the SMTP client used for delivery [7]. The interface does not claim success on its own; it relies on the backend response. A short undo grace period in the UI cannot recall a message after SMTP accepts it.

Chapter 4: Testing and results

4.1 Verification approach

The backend tests use FastAPI's test client with a fake AI service and mocked mail behavior. This keeps the route checks repeatable and avoids depending on a live Groq key or SMTP account. The frontend production build checks that the application can be bundled for deployment. A separate local Groq draft request was also exercised when credentials were available.

Check	Result
Frontend production build (npm run build)	Passed.
Backend route and validation tests (pytest -q)	11 tests passed.
Draft request schema and invalid tone/body	Required fields are documented; invalid payloads return 422.
Missing AI key	AI endpoint returns 503 with a safe configuration message.
Local Groq draft request	A generic draft request returned successfully with the configured model.
SMTP behavior	Dry-run and delivery error paths are covered with mocked responses.

4.2 Results and observations

The checks show that the main application paths are connected: the frontend can be built, the backend validates the request schemas, and route behavior can be tested without sending mail. A live draft call also confirmed the configured Groq path at the time it was run. These checks do not measure classification accuracy across a representative email dataset, and they do not verify delivery to a real mailbox.

For the draft endpoint, the tone values are case-sensitive and must be Professional, Formal, or Friendly. A lowercase value such as friendly is rejected with HTTP 422, which is expected request validation rather than an AI provider error. A missing key is reported as HTTP 503, while provider authentication, rate-limit, timeout, and model errors are handled separately.

Chapter 5: Limitations and conclusion

5.1 Current limits

The application is a demo inbox, not a mailbox client. It has no Gmail or Outlook integration, no multi-user authentication, and no server-side mailbox database. Message text and drafts persist in browser localStorage, which is not encrypted. When AI features are used, the relevant email content is sent to the configured Groq service. Anyone using the app should have permission to share that content with the provider.

The AI can misread a message, overlook context, or produce a reply that needs correction. Priority explanations are suggestions rather than verified facts. SMTP delivery also depends on external configuration, and the current UI undo delay is not a server-side cancellation queue. These limits make user review and careful configuration part of the current design.

5.2 Future work

The clearest next step is an approved OAuth connection to a real mailbox, paired with server-side user accounts and protected message storage. A durable send queue would make cancellation meaningful. Before wider use, the project would also benefit from retention controls, broader evaluation on representative messages, monitoring for provider failures, and deployment-specific security testing.

Conclusion

This project brings inbox triage and reply drafting into one approachable workflow. It gives a person a quicker way to see what may need attention and a place to begin writing, while leaving the final review and send decision with them. The current code is a working local prototype; connecting it safely to real mailboxes and protecting stored user data are the main steps still ahead.

References

The framework and provider documentation below supports the technical descriptions in this report. Project behavior and test results come from the source files listed in reference 8.

[1] FastAPI. Request Body
https://fastapi.tiangolo.com/tutorial/body/ (accessed 29 October 2026).

[2] React. Quick Start
https://react.dev/learn (accessed 29 October 2026).

[3] Vite. Server Options: server.proxy
https://vite.dev/config/server-options (accessed 29 October 2026).

[4] LangChain. ChatGroq integration
https://docs.langchain.com/oss/python/integrations/chat/groq (accessed 29 October 2026).

[5] Pydantic. Models
https://pydantic.dev/docs/validation/latest/concepts/models/ (accessed 29 October 2026).

[6] Groq. Quickstart
https://console.groq.com/docs/quickstart (accessed 29 October 2026).

[7] Python Software Foundation. smtplib - SMTP protocol client
https://docs.python.org/3/library/smtplib.html (accessed 29 October 2026).

[8] Project source. README.md; backend/app/main.py, config.py, schemas.py, services/ai_service.py, services/mail_service.py, tests/test_api.py; frontend/src and frontend/vite.config.js.